NHH

SCHOOL EXAMINATION

BAN404

Spring, 2025

Date: 02.06.2025

Time: 9.00-15.00

Number of hours: 6

An invigilator can contact course responsible by phone: 47078891

SUPPORT MATERIALS PERMITTED DURING THE EXAMINATION:

Calculator

All written support material permitted (category III)

Number of pages, including front page: 5

Number of attachments: 1, The file "data_task1.csv".

Instructions and recommendations

General recommendations: Be pragmatic if you run into difficulties. You need to find models that can answer the questions asked. Don´t let the perfect model stand in the way for a good one! Also, plan your work so that you do not use all time to solve one task.

R coding can be done in different ways, e.g. using base R or the tidyverse approach. Given a correct and clearly written result there is no preference for any particular approach.

Throughout the exam you will work with training and test data and you will have to make appropriate choices on when to use one or the other. In cases when you think there are more than one valid choice on working with the training, test or the full dataset, explain your choice!

Tasks

1. (50 points) This task is about some of the methodological topics treated in the course.

   (a) (10 points) Explain what the following R-functions are doing. Which method, treated in the course, is this a version of?

```r
# X is a n x p matrix with data
# y is an n x 1 vector with data

f <- function(b1,X,y,la)
{
  fval <- sum((y-X%*%b1)^2) + la*sum(b1^2)
  return(fval)
}

g <- function(X,y,la)
{
  q <- ncol(X)
  b0 <- mean(y)
  yd <- y - b0
  Xd <- X
  for(k in 1:q) Xd[,k] <- X[,k] - mean(X[,k])
  b1start <- rep(0,q)
  opt <- nlminb(b1start,f,X=Xd,y=yd,la=la)
  b1 <- opt$par
  return(b1)
}
```

Solution:

```r
# X is a n x p matrix with data
# y is an n x 1 vector with data

f <- function(b1,X,y,la) # Fitting function for ridge regression
{
  fval <- sum((y-X%*%b1)^2) + la*sum(b1^2)
  # Compute residual sum of squares plus penalty
  # for large sum of reg.coef.
  return(fval)
  # Return function value
}

g <- function(X,y,la)
  # Estimating ridge regression for a given lambda
{
  q <- ncol(X) # Number of variables
  b0 <- mean(y) # Compute intercept
  yd <- y - b0 # Demean y
  Xd <- X # Initialize matrix for demeaned x-variables
  for(k in 1:q) Xd[,k] <- X[,k] - mean(X[,k])
  # Demean all x-variables
  b1start <- rep(0,q)
  # Starting value for minization
  opt <- nlminb(b1start,f,X=Xd,y=yd,la=la)
  # Minimize f with respect to b1 for a given lambda
  b1 <- opt$par
  # Extract the coefficients giving the minimum value
  return(b1)   # Return the coefficients
}
```

See comments in R code. This is a version of Ridge regression where the X-variables are demeaned but not standardized.

(b) (10 points) Consider the dataset in the file `data_task1.csv`. Extract the X- and y-values by

```r
# Read data
Xy <- read.csv2("data_task1.csv")
X <- as.matrix(Xy[,-1])
y <- as.matrix(Xy[,1])
```

These are 100 observations of 20 independent variables and 1 dependent variable. Fit the model by ordinary least squares (OLS) (with the `lm`-function) and by using

g with `la=1000`. Comment on the difference between the results. What is the explanation for the difference? Also, fit the model using g and let `la=0`.

Solution:

```r
b1ols <- lm(y~X)$coef[-1]
# OLS regression, do not save intercept
b1g0 <- g(X,y,0)
b1g1000 <- g(X,y,1000)
cbind(b1ols,b1g0,b1g1000)
```

```
##           b1ols       b1g0    b1g1000
## XX1   26.198055  26.198039  22.631336
## XX2  -17.161224 -17.161120  -5.469236
## XX3   21.793449  21.793321   9.705557
## XX4   61.559375  61.559296  39.410146
## XX5    8.023438   8.023527  11.187845
## XX6  -18.216741 -18.216696 -10.011058
## XX7   24.787363  24.787353  19.775821
## XX8   51.743651  51.743591  33.992245
## XX9   35.769806  35.769847  35.681847
## XX10 -26.239930 -26.239889 -12.822297
## XX11 -32.985179 -32.985148 -22.476505
## XX12 -12.264092 -12.264130  -4.630471
## XX13   3.535228   3.535291  12.157542
## XX14 -43.238530 -43.238408 -27.213784
## XX15  26.127038  26.126860   6.835591
## XX16  32.451475  32.451456  26.967940
## XX17 -10.730619 -10.730418   2.573308
## XX18 -41.533946 -41.533907 -23.539140
## XX19   3.477019   3.477017  -2.526572
## XX20 -24.029345 -24.029331 -13.394105
```

```r
sum(b1ols^2)
```

```
## [1] 18187.35
```

```r
sum(b1g1000^2)
```

```
## [1] 8334.522
```

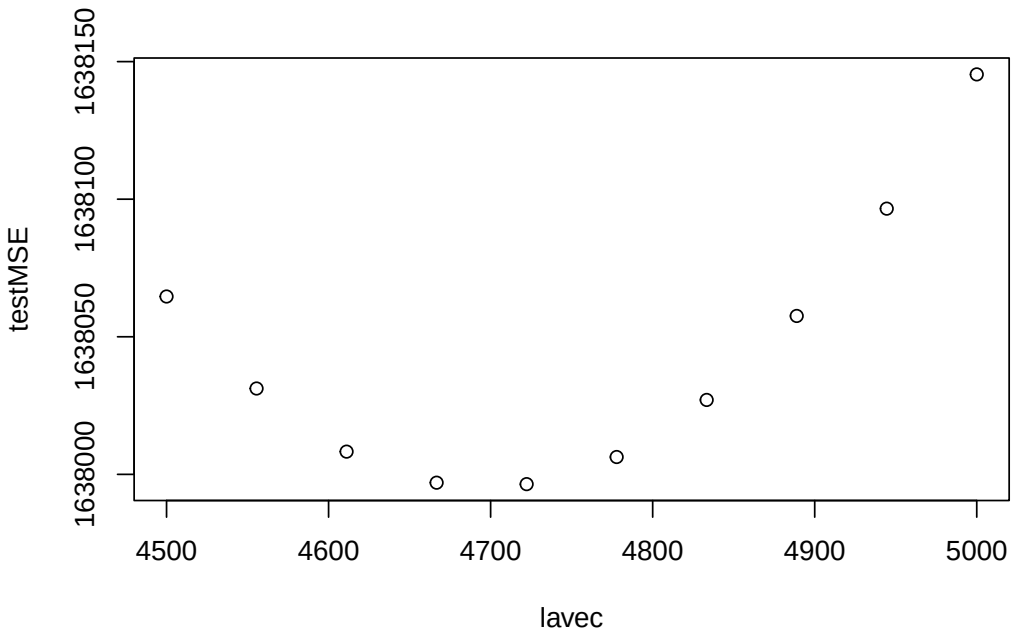
The coefficients in the third column are estimated with a restriction on the sum of the squared coefficients. This is indicated by the fact that the sum of the squared coefficients are smaller when the g-function is used with `la=1000`.

(c) (10 points) Explain what the following R-function does and use it to determine an optimal `la` for this dataset. Is it necessary to demean the x-variables? If so, why?

```
loo <- function(la,X,y)
{
  n <- nrow(X)
  q <- ncol(X)
  b0 <- mean(y)
  yd <- y - b0
  Xd <- X
  for(k in 1:q) Xd[,k] <- X[,k] - mean(X[,k])
  ydpred <- matrix(0,n,1)
  for(i in 1:n)
  {
    b1 <- g(X=X[-i,],y=y[-i],la)
    ydpred[i] <- Xd[i,]%*%b1
  }
  testMSE <- mean((yd-ydpred)^2)
  return(testMSE)
}
```

Solution:

```
lavec <- seq(4500,5000,len=10)
nla <- length(lavec)
testMSE <- matrix(0,nla,1)
for(i in 1:nla) testMSE[i] <- loo(lavec[i],X,y)
plot(lavec,testMSE)
```




```
lavec[which.min(testMSE)]
```

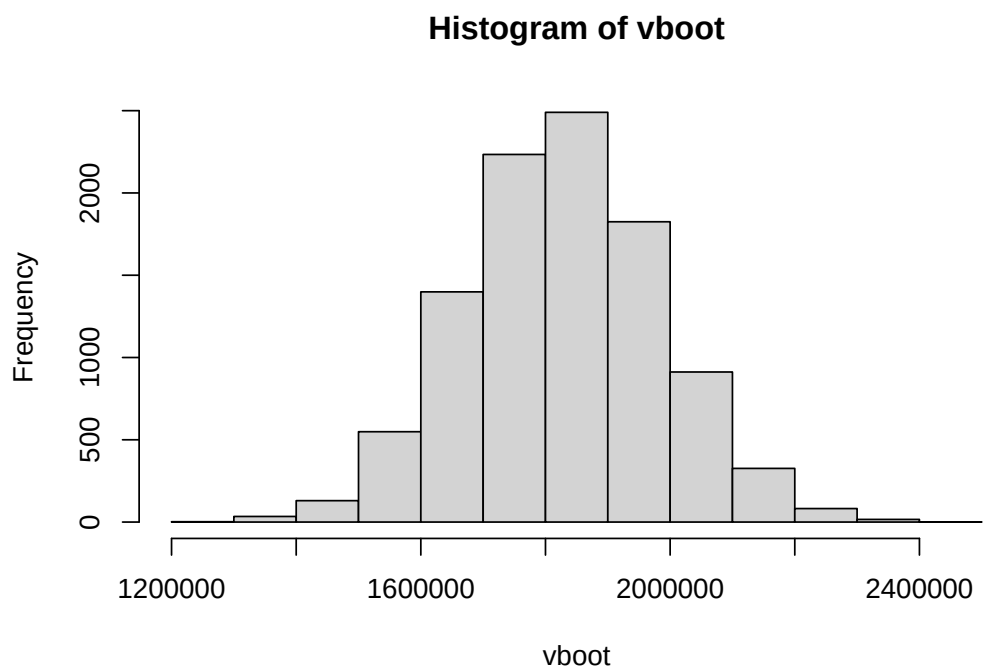
```
## [1] 4722.222
```

The code is using leave-one-out cross-validation to determine `la`. By starting computing `loo` for 10 equidistant values between 0 and 10000 and plotting them I concluded that the optimal value is somewhere between 4000 and 6000. 10 values between 4000 and 6000 was therefore investigated and the conclusion was that the optimum is somewhere between 4500 and 5000. By checking 10 values between 4500 and 5000 an approximate optimal `la` of 4722 was found. It is necessary to demean the x-variables since they are demeaned in the g-function. The coefficients in `b1` must, therefore, be multiplied by the demeaned values.

(d) (10 points) Use the bootstrap method to estimate the sampling distribution of the sample variance of the variable `y` in `data_task1.csv`, $S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2$. Show the result as a histogram. Also, use the bootstrap to compute a 95% confidence interval of the variance of `y`, $\sigma_y^2 = \mathrm{Var}(y)$.

Solution:

```r
n <- length(y)
B <- 10000
vboot <- matrix(0,B,1)
for(b in 1:B)
{
  yboot <- sample(y,size=n,replace=TRUE)
  vboot[b] <- var(yboot)
}
hist(vboot)
```




```r
quantile(vboot,probs = c(0.025,0.975))
```

```
##    2.5%   97.5%
## 1522449 2134932
```

(e) (10 points) There might be one or more variables that have non-linear relationships to y. Try to identify such variable(s). Can you improve the in-sample (training) MSE by using a generalized additive model (GAM)? Compare the training MSE with at least one GAM. Also, use these MSE's to compute the share of explained variation, R^2, for the linear model estimated by OLS and for the GAM-model. If you did not obtain any MSE's from the models, or do not trust them, you can use $\text{MSE}_{\text{gam}} = 110000$ and $\text{MSE}_{\text{ols}} = 121000$.

Solution:

```
library(gam)
formula1 <- "y~X1+X2+s(X3)+"
for(i in 4:20) formula1 <- paste(formula1,"+X",i,sep="")
formula1 <- formula(formula1)
gam1 <- gam(formula1,data=Xy)
ols <- lm(y~.,data=Xy)
MSE_gam <- mean(residuals(gam1)^2)
MSE_ols <- mean(residuals(ols)^2)
MSE_all <- mean((y-mean(y))^2)
R2_gam <- 1-MSE_gam/MSE_all
R2_ols <- 1-MSE_ols/MSE_all
R2_gam
```

```
## [1] 0.3975391
```

```
R2_ols
```

```
## [1] 0.3392508
```

Modelling the non-linearity in X3 by a smoothing spline improves the training MSE (and increases R^2).

2. (50 points) In this task, the dataset `dataOhlsson` in the R-package `insuranceData` shall be analyzed. Use `help(dataOhlsson)` to read about the dataset but note that the variable `kon` is gender and not age as said in the help-function.

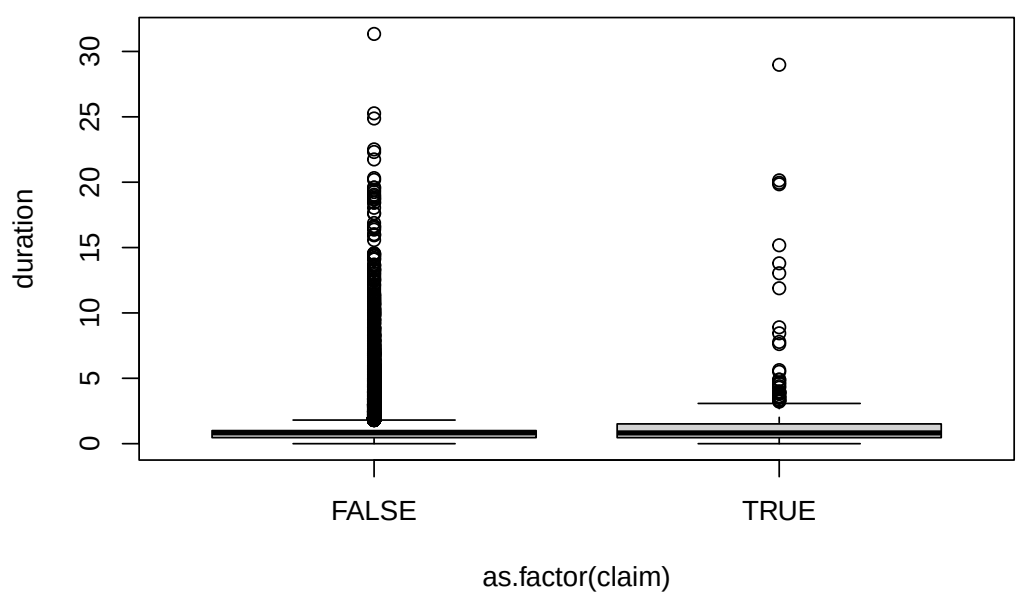
   (a) (10 points) Create a new variable called `claim` by recoding `antskad` to a binary variable (or a factor). Let it be equal to one ("yes") if `antskad`≥ 1 and zero ("no") otherwise.

```
library(insuranceData)
data("dataOhlsson")
ydata=dataOhlsson
ydata$claim=ydata$antskad>=1
```

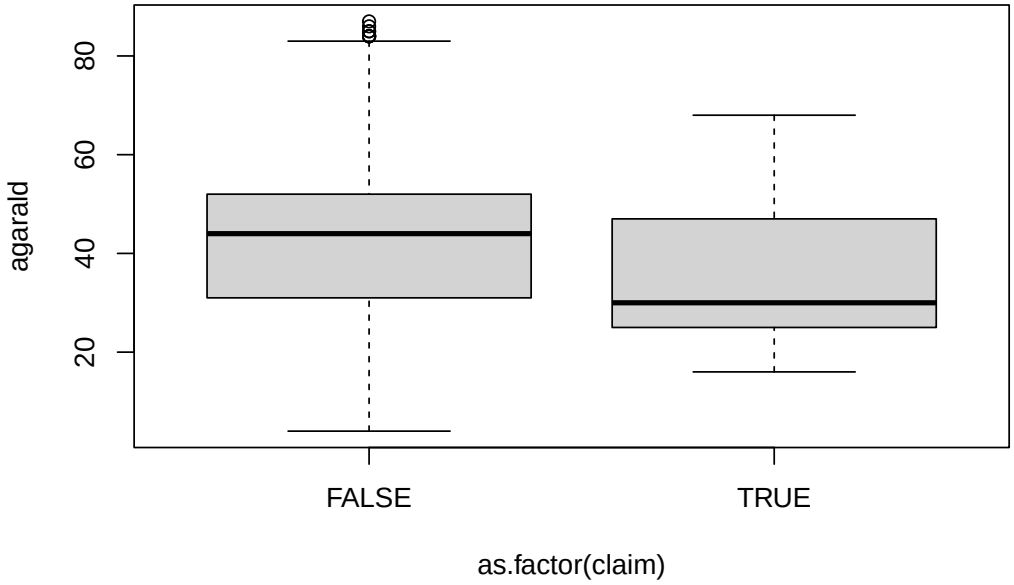
   You are going to predict `claim` so remove variables that cannot plausibly be assumed to be available when a prediction is made and explain your reasoning. Split the data into test and training data. Use descriptive statistics and graphs in order to find potentially useful predictors.

   Solution:

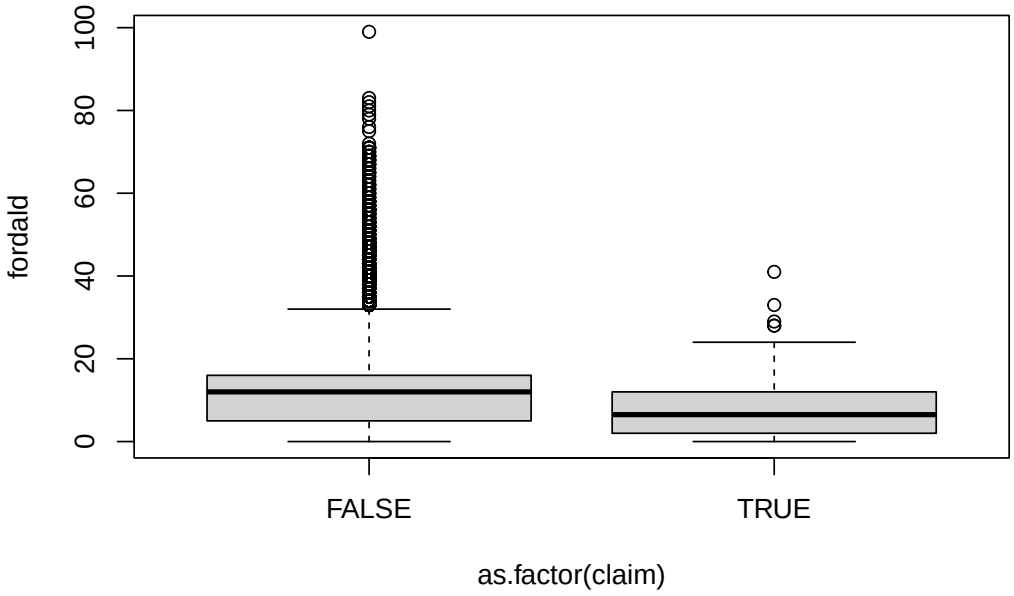
```r
library(insuranceData)
data("dataOhlsson")
ydata=dataOhlsson
ydata$claim=ydata$antskad>=1
ydata=ydata[,!(names(ydata) %in% c("antskad","skadkost"))]
ydata$zon=as.factor(ydata$zon)
ydata$bonuskl=as.factor(ydata$bonuskl)
ydata$mcklass=as.factor(ydata$mcklass)
set.seed(123)
n <- nrow(ydata)
ntrain <- floor(n/2)
ind <- sample(1:n,size=ntrain)
train <- ydata[ind,]
test <- ydata[-ind,]
# Numerical predictors
plot(duration~as.factor(claim),data=train)
```




```r
# Not visible difference
plot(agarald~as.factor(claim),data=train)
```




```r
# Lower age for claim=1
plot(fordald~as.factor(claim),data=train)
```



```r
# Lower vehicle age for claim=1
# Categorical predictors
prop.table(table(train$kon,train$claim),margin=1)
```

```
##
##          FALSE       TRUE
##   K 0.99426582 0.00573418
##   M 0.98882845 0.01117155
```

```r
# Men higher freq of claim=1
prop.table(table(train$bonuskl,train$claim),margin=1)
```

```
##
##           FALSE        TRUE
##   1 0.990488007 0.009511993
##   2 0.993548387 0.006451613
##   3 0.990678707 0.009321293
##   4 0.989975804 0.010024196
##   5 0.992226972 0.007773028
##   6 0.992891882 0.007108118
##   7 0.984809561 0.015190439
```

```r
# Highest class more claim=1
prop.table(table(train$zon,train$claim),margin=1)
```

```
##
##           FALSE        TRUE
##   1 0.979530123 0.020469877
##   2 0.985333783 0.014666217
##   3 0.990398237 0.009601763
##   4 0.993164455 0.006835545
##   5 0.996560619 0.003439381
##   6 0.995826813 0.004173187
##   7 0.994285714 0.005714286
```

```r
# zon=1 higher more claim=1
prop.table(table(train$mcklass,train$claim),margin=1)
```

```
##
##           FALSE        TRUE
##   1 0.994227994 0.005772006
##   2 0.988734510 0.011265490
##   3 0.991271427 0.008728573
##   4 0.992263056 0.007736944
```

```
##   5 0.988472127 0.011527873
##   6 0.980573324 0.019426676
##   7 0.990000000 0.010000000
```

```r
# No particular pattern
```

I do not observe any very remarkable patterns. Two variables worth special mentioning are `agarald` and `fordald` which seem to have quite different distributions for `claim=0` and `claim=1`.

(b) (10 points) Fit a logistic regression model with all variables and one with a selection of variables. The selection can be based on your arguments in task 2a and by the result of the model with all variables.

Solution:

```r
logreg1 <- glm(claim~.,data=train,family = binomial())
summary(logreg1)
```

```
##
## Call:
## glm(formula = claim ~ ., family = binomial(), data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.305767   0.362613  -6.359 2.03e-10 ***
## agarald     -0.047866   0.005008  -9.558  < 2e-16 ***
## konM         0.676923   0.200700   3.373 0.000744 ***
## zon2        -0.281711   0.154657  -1.822 0.068528 .
## zon3        -0.786883   0.170424  -4.617 3.89e-06 ***
## zon4        -1.150491   0.164125  -7.010 2.39e-12 ***
## zon5        -1.877668   0.514098  -3.652 0.000260 ***
## zon6        -1.464143   0.372702  -3.928 8.55e-05 ***
## zon7        -1.283589   1.013377  -1.267 0.205282
## mcklass2     0.398606   0.292508   1.363 0.172971
## mcklass3    -0.046144   0.255342  -0.181 0.856592
## mcklass4    -0.095665   0.269807  -0.355 0.722914
## mcklass5     0.249489   0.259756   0.960 0.336816
## mcklass6     0.704384   0.254898   2.763 0.005720 **
## mcklass7    -0.306640   0.556089  -0.551 0.581344
## fordald     -0.079741   0.009378  -8.503  < 2e-16 ***
## bonuskl2    -0.350021   0.223444  -1.566 0.117236
```

```
## bonuskl3     0.010843   0.216963   0.050 0.960141
## bonuskl4     0.005628   0.225679   0.025 0.980105
## bonuskl5    -0.254256   0.258108  -0.985 0.324586
## bonuskl6    -0.316552   0.263901  -1.200 0.230329
## bonuskl7     0.413306   0.161584   2.558 0.010533 *
## duration     0.181016   0.021950   8.247  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3717.9  on 32273  degrees of freedom
## Residual deviance: 3313.7  on 32251  degrees of freedom
## AIC: 3359.7
##
## Number of Fisher Scoring iterations: 8
```

```r
logreg2 <- glm(claim~agarald+kon+zon+fordald+duration,
               data=train,family=binomial())
summary(logreg2)
```

```
##
## Call:
## glm(formula = claim ~ agarald + kon + zon + fordald + duration,
##     family = binomial(), data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.229932   0.271093  -8.226  < 2e-16 ***
## agarald     -0.045544   0.004729  -9.631  < 2e-16 ***
## konM         0.692202   0.199399   3.471 0.000518 ***
## zon2        -0.250733   0.153682  -1.631 0.102785
## zon3        -0.732927   0.168951  -4.338 1.44e-05 ***
## zon4        -1.131166   0.161665  -6.997 2.62e-12 ***
## zon5        -1.754933   0.513016  -3.421 0.000624 ***
## zon6        -1.366306   0.371346  -3.679 0.000234 ***
## zon7        -1.104859   1.011557  -1.092 0.274730
## fordald     -0.078598   0.009020  -8.714  < 2e-16 ***
## duration     0.206305   0.020626  10.002  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3717.9  on 32273  degrees of freedom
## Residual deviance: 3371.5  on 32263  degrees of freedom
## AIC: 3393.5
##
## Number of Fisher Scoring iterations: 8
```

(c) (10 points) Evaluate the predictions of the model in 2b on test data and with an appropriate (in)accuracy measure. Hint: You will first have to determine a threshold to convert from a probability to a prediction of the type `claim=0` or `claim=1`. You can experiment here and you do not need to do a formal cross-validation to find this threshold.

Solution:

```r
# Determine threshold
prob1 <- predict(logreg1,newdata=train,type="response")
pred1 <- prob1>0.01
prob2 <- predict(logreg2,newdata=train,type="response")
pred2 <- prob2>0.01
prop.table(table(train$claim,pred1),margin=1)
```

```
##        pred1
##            FALSE     TRUE
##   FALSE 0.689355 0.310645
##   TRUE  0.257485 0.742515
```

```r
prop.table(table(train$claim,pred2),margin=1)
```

```
##        pred2
##             FALSE      TRUE
##   FALSE 0.6719161 0.3280839
##   TRUE  0.2814371 0.7185629
```

```r
# Evaluate predictions
prob1 <- predict(logreg1,newdata=test,type="response")
pred1 <- prob1>0.01
prob2 <- predict(logreg2,newdata=test,type="response")
pred2 <- prob2>0.01
prop.table(table(test$claim,pred1),margin=1)
```

```
##        pred1
```

```
##              FALSE      TRUE
##   FALSE 0.6861732 0.3138268
##   TRUE  0.2827381 0.7172619
```

```r
prop.table(table(test$claim,pred2),margin=1)
```

```
##        pred2
##              FALSE      TRUE
##   FALSE 0.6684201 0.3315799
##   TRUE  0.3154762 0.6845238
```

The predictions are quite weak but with a threshold of 0.01 around 70% of the observations are correctly classified both in the claim- and in the no claim categories.

(d) (10 points) Fit a random forest and evaluate the predictions. Compare with the logistic model.

Solution:

```r
library(randomForest)
rf <- randomForest(as.factor(claim)~.,data=train,mtry=7)
prob_rf <- predict(rf,newdata = test, type="prob")
pred_rf <- prob_rf[,2] > 0.02
round(prop.table(table(test$claim,pred_rf),margin=1),3)
```

```
##        pred_rf
##         FALSE  TRUE
##   FALSE 0.835 0.165
##   TRUE  0.568 0.432
```

Threshold can also here be found by experimenting on training data. Better prediction for `claim=0` but worse for `claim=1` compared to logistic regression.

(e) (10 points) Fit a boosted tree and evaluate the predictions. Compare with the logistic model and the random forest.

Solution:

```r
library(gbm)
bt <- gbm(as.numeric(claim)~.,data=train,
          distribution = "bernoulli")
prob_bt <- predict(bt,newdata = test,type="response")
pred_bt <- prob_bt>0.01
round(prop.table(table(test$claim,pred_bt),margin=1),3)
```

```
##        pred_bt
##         FALSE  TRUE
##   FALSE 0.669 0.331
##   TRUE  0.295 0.705
```

Threshold can also here be found by experimenting on training data. Very similar results as with logistic regression.